

- Documentation
 - Quick Links
 - Getting Started
 - 1. Setup Environment
 - 2. Collect Data
 - 3. Run Backtest
 - 4. Player Filtering (Memory Optimization)
 - Deployment Options
 - Integrated vs Separated Architecture
 - Cloud Training
 - Documentation Structure
 -  Setup & Configuration
 -  Backtesting & Evaluation
 -  Feature Engineering
 -  User Interfaces
 -  Performance Optimization
 -  Deployment & Infrastructure
 - Project Documentation
 - Data Flow Architecture
 - Key Concepts
 - Walk-Forward Backtesting
 - Benchmark Comparison
 - Feature Configuration
 - Model Persistence
 - Common Workflows
 - Development Workflow
 - Production Workflow
 - Performance Benchmarks
 - Current Results (2025-02-05)
 - Target Metrics
 - Troubleshooting
 - Common Issues
 - Debug Mode
 - Additional Resources
 - External Links
 - Internal References
 - Contributing

- [Support](#)

Documentation

Complete documentation for the NBA DFS machine learning pipeline.

Quick Links

- [Scripts Guide](#) - Complete guide to all scripts with usage examples
- [Feature Configuration](#) - YAML-based feature pipeline configuration
- [Local Separated Setup](#) - Separated architecture for local development
- [Google Colab Setup](#) - Cloud training on Google Colab
- [Project Overview](#) - Architecture, modules, and development workflow
- [Scripts README](#) - Detailed script documentation

Getting Started

1. Setup Environment

```
# Install dependencies
pip install -r requirements.txt

# Configure API key
echo "TANK01_API_KEY=your_key_here" > .env
```

2. Collect Data

```
# Collect historical game data
python scripts/collect_games.py --start-date 20241201 --end-date 20241231

# Collect DFS salaries
python scripts/collect_dfs_salaries.py --start-date 20241201 --end-date 20241231
```

3. Run Backtest

```
# Single day backtest
python scripts/run_backtest.py --test-start 20250205 --test-end 20250205

# Multi-day with per-player models
python scripts/run_backtest.py --test-start 20250201 --test-end 20250207 --per-player
```

4. Player Filtering (Memory Optimization)

Filter players before loading training data to reduce memory usage and speed up backtests:

```
# Filter by minimum salary
python scripts/run_backtest.py --test-start 20250205 --test-end 20250215 \
  --filter-salary-min 5000

# Filter by player names
python scripts/run_backtest.py --test-start 20250205 --test-end 20250215 \
  --filter-player-names "LeBron James,Stephen Curry,Kevin Durant"

# Filter by player IDs from CSV
python scripts/run_backtest.py --test-start 20250205 --test-end 20250215 \
  --filter-player-ids-csv data/elite_players.csv

# Combine multiple filters
python scripts/run_backtest.py --test-start 20250205 --test-end 20250215 \
  --filter-salary-min 7000 --filter-injury-exclude OUT,GTD
```

Benefits:

- Pre-scan identifies filtered players across all test slates
- Only loads historical data for filtered players
- Benchmark fitted only on filtered subset
- Reduced memory footprint (e.g., 2 players vs. 500 players)
- Faster feature engineering with smaller datasets

Deployment Options

Integrated vs Separated Architecture

Integrated (default): Code and data in same directory. Simple setup.

Separated: Code and data in different locations. Clean git repo, flexible storage.

```
# Separated architecture usage
python scripts/run_backtest.py \
  --data-dir D:\NBA_Data \
  --test-start 20250201 \
  --test-end 20250207 \
  --per-player
```

See [LOCAL_SEPARATED_SETUP.md](#) for complete guide.

Cloud Training

Free machine during training with Google Colab:

- **Colab Free:** \$0/month, ~21 min/slate
- **Colab Pro:** \$10/month, ~10.4 min/slate (recommended)
- **Colab Pro+:** \$50/month, ~5.2 min/slate

See [COLAB_SETUP.md](#) for setup instructions.

Documentation Structure

Setup & Configuration

Directory: **setup/**

- [SCRIPTS_GUIDE.md](#) - Comprehensive guide to all scripts
 - Data collection scripts
 - Model training and evaluation
 - Command examples and arguments
 - Troubleshooting and best practices

- [LOCAL_SEPARATED_SETUP.md](#) - Separated architecture guide
 - Directory structure and benefits
 - Setup instructions
 - Path resolution and configuration
 - Storage recommendations
- [COLAB_SETUP.md](#) - Google Colab training guide
 - Colab tier comparison and pricing
 - Setup and data sync instructions
 - Configuration and optimization
 - Best practices for cloud training



Backtesting & Evaluation

Directory: [backtesting/](#)

- [BACKTESTING_PIPELINE_FLOW.md](#) - Complete pipeline flow & logic
- [BACKTESTING_DATA_EXAMPLES.md](#) - Data transformation examples
- [VALID_BACKTEST_CONFIGURATIONS.md](#) - Working configurations
- [BACKTEST_TROUBLESHOOTING.md](#) - Common issues & solutions
- [CHECKPOINT_RESUME.md](#) - Resume interrupted backtests
- [LINEUP_GENERATION.md](#) - Optimize lineups from predictions



Feature Engineering

Directory: [features/](#)

- [FEATURE_CONFIG_INTEGRATION.md](#) - YAML-based feature configuration
 - Configuration system
 - Rolling statistics transformers
 - EWMA transformers
 - Custom feature creation
- [OPPONENT_FEATURES.md](#) - Opponent matchup features
- [PLAYER_FILTERING.md](#) - Player filtering strategies



User Interfaces

Directory: [interfaces/](#)

- [PANEL_INTERFACE.md](#) - Interactive web-based control center
 - Real-time backtest monitoring
 - Configuration management
 - Live log streaming



Performance Optimization

Directory: [optimization/](#)

- [GPU_OPTIMIZATION_GUIDE.md](#) - GPU acceleration implementation
 - Feature preprocessing cache
 - GPU batch training system
 - Performance benchmarks
- [BAYESIAN_OPTIMIZATION.md](#) - Hyperparameter optimization



Deployment & Infrastructure

Directory: [deployment/](#)

- [SAGEMAKER_DEPLOYMENT.md](#) - AWS SageMaker deployment

Project Documentation

- [../CLAUDE.md](#) - Main project documentation
 - Architecture overview
 - Module descriptions
 - Development workflows
 - Usage examples
- [../scripts/README.md](#) - Scripts reference
 - Detailed script documentation

- API usage estimation
- Output file formats
- Error handling

Data Flow Architecture

```
Parse error on line 2:
...ph LR      subgraph "Data Collection"
-----^
Expecting 'SEMI', 'NEWLINE', 'SPACE', 'EOF', 'GRAPH', 'DIR',
'TAGEND', 'TAGSTART', 'UP', 'DOWN', 'subgraph', 'end', 'SQE',
'PE', '-)', 'DIAMOND_STOP', 'MINUS', '--', 'ARROW_POINT',
'ARROW_CIRCLE', 'ARROW_CROSS', 'ARROW_OPEN',
'DOTTED_ARROW_POINT', 'DOTTED_ARROW_CIRCLE',
'DOTTED_ARROW_CROSS', 'DOTTED_ARROW_OPEN', '==',
'THICK_ARROW_POINT', 'THICK_ARROW_CIRCLE', 'THICK_ARROW_CROSS',
'THICK_ARROW_OPEN', 'PIPE', 'STYLE', 'LINKSTYLE', 'CLASSDEF',
'CLASS', 'CLICK', 'DEFAULT', 'NUM', 'PCT', 'COMMA', 'ALPHA',
'COLON', 'BRKT', 'DOT', 'PUNCTUATION', 'UNICODE_TEXT', 'PLUS',
'EQUALS', 'MULT', got 'STR'
```

Key Concepts

Walk-Forward Backtesting

Walk-forward validation tests models on unseen future data:

1. Train on historical data (e.g., season to date)
2. Predict next slate
3. Evaluate against actual results
4. Move forward to next slate

Script: `run_backtest.py`

Benchmark Comparison

Compare model predictions against season average baseline:

- Statistical significance testing (paired t-test)

- Effect size calculation (Cohen's d)
- Salary tier analysis

Scripts: `run_backtest.py`, `evaluate_with_benchmark.py`

Feature Configuration

YAML-based feature pipeline for reproducibility:

- `default_features.yaml` - Full 147 features
- `base_features.yaml` - Minimal 6 features for quick iteration

Directory: `config/features/`

Model Persistence

Save trained models for reuse:

- Per-player models: One model per player
- Slate-wide models: Single model per date
- Metadata JSON files with training info

Directory: `data/models/`

Common Workflows

Development Workflow

1. Data Collection

```
python scripts/collect_games.py --start-date 20241001 --end-date 20241231
python scripts/collect_dfs_salaries.py --start-date 20241001 --end-date
20241231
```

2. Feature Engineering

- Edit `config/features/*.yaml`

- Test with `base_features` for speed
- Switch to `default_features` for full pipeline

3. Hyperparameter Optimization

```
python scripts/optimize_xgboost_hyperparameters.py --n-trials 50
```

4. Backtesting

```
python scripts/run_backtest.py --test-start 20250101 --test-end 20250131
```

5. Analysis

- Review `data/backtest_results/summary_*.txt`
- Analyze `data/backtest_results/tier_comparison_*.csv`
- Examine predictions in `data/models/per_slate/*.parquet`

Production Workflow

1. Collect Latest Data

```
python scripts/collect_games.py --start-date TODAY --end-date TODAY  
python scripts/collect_dfs_salaries.py --start-date TODAY --end-date TODAY  
python scripts/collect_depth_charts.py --all-teams
```

2. Load Trained Models

- Per-player: `data/models/per_player/*.pkl`
- Slate-wide: `data/models/per_slate/*_slate_model.pkl`

3. Generate Predictions

- Use `WalkForwardBacktest` or custom inference pipeline
- Apply models to today's slate features

4. Optimize Lineups

- Use `LinearProgramOptimizer` with predictions

- Apply DraftKings constraints
- Export optimal lineups

Performance Benchmarks

Current Results (2025-02-05)

Elite Players (\$8k+):

- MAPE: 32.9% (near 30% target)
- Strong correlation: 0.728

Overall:

- MAPE: 81.18%
- Coverage: 96.4% of players
- Issue: Low-output player MAPE inflation

Benchmark Comparison:

- Model slightly worse than season average baseline
- Statistical significance: $p < 0.05$ (small effect size)
- Improvement needed in mid-tier players

Target Metrics

- Elite players: <30% MAPE
- High salary (\$6k-8k): <40% MAPE
- Overall: <60% MAPE
- Correlation: >0.7

Troubleshooting

Common Issues

API Rate Limits

- Free tier: 1000 requests/month
- Solution: Collect smaller date ranges
- Monitor: Check console output for remaining requests

Memory Issues

- Per-player models use significant memory
- Solution: Use `--no-save-models` flag
- Solution: Process smaller date ranges

Missing Dependencies

- Run: `pip install -r requirements.txt`
- Check: Python version ≥ 3.8

Database Errors

- Ensure data collection completed successfully
- Check: `data/inputs/` directories have .parquet files
- Run: `python scripts/load_games_to_db.py` to rebuild

Debug Mode

Enable verbose logging:

```
python scripts/run_backtest.py --test-start 20250205 --test-end 20250205 --verbose
```

Additional Resources

External Links

- [Tank01 Fantasy Stats API](#) - Data source
- [RapidAPI](#) - API marketplace
- [XGBoost Documentation](#) - Model documentation
- [Optuna Documentation](#) - Hyperparameter optimization

Internal References

- Configuration files: [config/](#)
- Source code: [src/](#)
- Tests: [tests/](#)
- Notebooks: [notebooks/](#)

Contributing

When adding new features:

1. Document in appropriate section
2. Add examples to [SCRIPTS_GUIDE.md](#)
3. Update [./CLAUDE.md](#) if architecture changes
4. Write tests in [tests/](#)

Support

For issues:

1. Check error messages in console output
2. Enable `--verbose` logging
3. Review relevant documentation section
4. Check existing issues in repository